



Manual de Evolução

Como acrescentar uma fonte, uma DAG, um modelo ou uma regra de qualidade à Plataforma de Dados MinC sem quebrar o que já existe, e quais decisões de arquitetura estão registradas e não devem ser refeitas.

Universidade de Brasília

Lab Livre · Faculdade do Gama, UnB

Ministério da Cultura

Gov Hub BR

Projeto de Pesquisa

Plataforma de Dados do Ministério da Cultura · Gov Hub BR

Meta 03 · Produto 3

Manuais de uso, manutenção e evolução.

Documento

Manual de Evolução



Índice

01	A quem se destina	4
	A arquitetura que sustenta a evolução	4
02	Acrescentar uma fonte	7
	Fase 1 · Cliente de acesso	7
	Fase 2 · DAG de ingestão	7
	Fase 3 · Modelos dbt	7
	Fase 4 · Verificações e testes	7
	Fase 5 · Documentação que acompanha	7
	Fase 6 · Coleta do site e abertura do PR	8
03	O que mantém o repositório coerente	9
	Ramificações	9
	Mensagens de commit	9
	Issues	10
	Segredos	10
	Onde uma skill nova deve morar	10
	O que é gerado e não se edita à mão	10
04	Decisões já tomadas	11
05	Dívidas conhecidas	12
	1. A automação não reprova o que deveria	12
	2. Há código sem uso em plugins	12





Este manual é para quem vai estender a plataforma: acrescentar uma fonte de dados, um pipeline, um modelo de transformação ou uma regra de qualidade.

O documento descreve o caminho já trilhado por cada uma das fontes em produção, e as convenções que mantêm o repositório coerente.

A arquitetura que sustenta a evolução

A plataforma é modular por camada e por domínio. Uma fonte nova entra pela ingestão, pausa no bronze e sobe até o gold sem tocar em nada que já existe, porque cada camada só conhece a anterior. É essa separação que torna a extensão previsível.

O caminho de um dado, da origem ao consumo

Cada camada conhece apenas a anterior — é essa separação que torna a extensão previsível

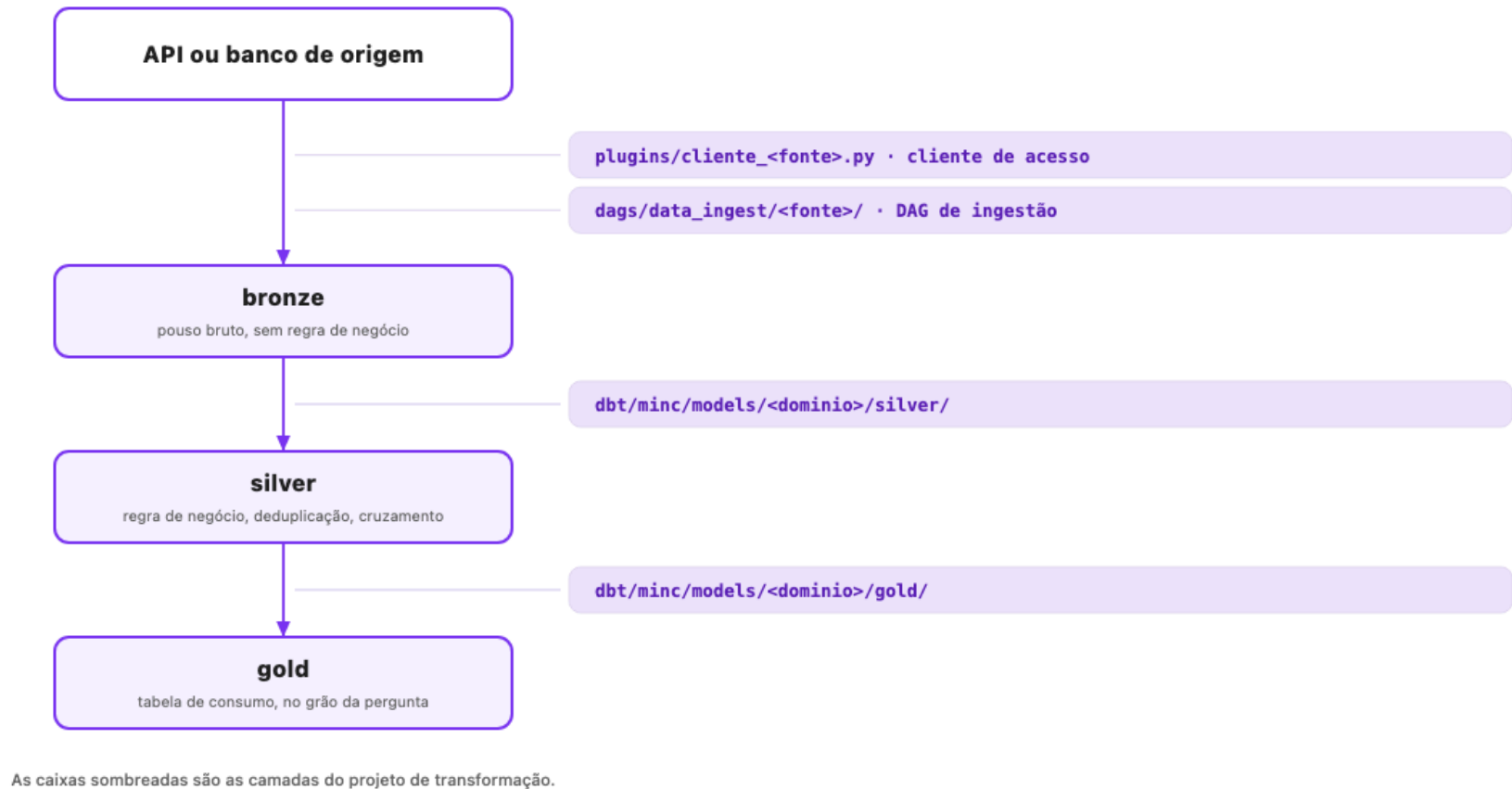


Figura 1. O caminho de um dado, da origem ao consumo. À direita, o lugar do repositório em que cada transição é implementada. As caixas sombreadas são as camadas do projeto de transformação.

EXISTE UMA SKILL QUE PERCORRE ESSE CAMINHO

O diretório [.claude/skills/govhub-pipeline-guide-minc/](https://github.com/claude/skills/govhub-pipeline-guide-minc/) descreve as seis fases de uma fonte nova, com a estrutura de arquivos esperada, os erros comuns e uma seção de diagnóstico por sintoma. Ela é versionada no repositório: quem clona a recebe junto.





Seis fases, na ordem. Pular uma delas costuma custar retrabalho na seguinte.

Fase 1 · Cliente de acesso

Um módulo `plugins/cliente_<fonte>.py` concentra autenticação, paginação e tratamento de erro da origem. Ele não conhece Airflow nem banco: recebe parâmetros e devolve dados. Essa separação é o que permite testar o cliente sem subir a orquestração.

Fase 2 · DAG de ingestão

Um arquivo por endpoint, em `dags/data_ingest/<fonte>/`. A DAG orquestra e persiste; ela não transforma. Nomes de schema e tabela vêm de `plugins/schemas_minc.py`, nunca de literal digitado no arquivo.

POR QUE O NOME DA TABELA NÃO PODE SER LITERAL

O cliente de banco executa `CREATE SCHEMA IF NOT EXISTS` a cada inserção. Um literal digitado errado não quebra a DAG: ele cria em silêncio um schema fora do padrão no banco. Foi assim que os nomes divergiram da especificação antes de serem centralizados, e é por isso que a centralização existe.

Fase 3 · Modelos dbt

Declare a tabela de pouso em `models/sources.yml`, escreva o modelo de bronze e suba pelas camadas. A materialização de cada camada já está definida em `dbt_project.yml`: não redefina modelo a modelo sem motivo registrado.

Fase 4 · Verificações e testes

Declare no `schema.yml` as verificações das cinco dimensões de qualidade que se aplicam ao modelo novo, e escreva teste em `tests/` para a lógica Python que não é trivial.

Fase 5 · Documentação que acompanha

Toda descrição de modelo e de coluna vive no `schema.yml` e em `descriptions.yml`. É de lá que saem o catálogo de metadados, o dicionário de dados e o catálogo externo. Um modelo sem descrição aparece como lacuna nos três.



Fase 6 · Coleta do site e abertura do PR

Quem altera um modelo ou uma DAG roda `make docs-collect` e inclui o acervo atualizado no mesmo PR. A publicação do site é automática, mas a coleta não: sem esse passo, o site segue descrevendo o estado anterior sem nenhum sinal de que está desatualizado.

ANTES DE CONSIDERAR PRONTO

Executar `make lint` e `make test` localmente. A automação de integração hoje confere apenas o SQL e não reprova o PR, de modo que a verificação de formatação, de estilo e de tipos depende dessa execução local.



O que mantém o repositório coerente



Ramificações

Dois formatos aceitos. O segundo é gerado automaticamente pelo fluxo de trabalho quando uma issue recebe a etiqueta *Código*.

Formato	Exemplo
<code><tipo>/<descricao-curta></code>	<code>feat/ingestao-salic</code>
<code><numero-da-issue>-<tipo>-<descricao></code>	<code>24-fix-dag-nota-de-credito</code>

Mensagens de commit

Conventional Commits, com os tipos definidos em `.github/TEMPLATES/COMMIT_TEMPLATE.md`.

Tipo	Quando usar
<code>feat</code>	Nova funcionalidade ou capacidade.
<code>fix</code>	Correção de erro.
<code>docs</code>	Alteração exclusiva de documentação.
<code>refactor</code>	Mudança que não corrige erro nem acrescenta funcionalidade.
<code>perf</code>	Mudança que melhora desempenho.
<code>test</code>	Adição ou correção de teste automatizado.
<code>build</code>	Mudança no sistema de construção ou em dependências.
<code>ci</code>	Mudança nos arquivos de integração contínua.
<code>chore</code>	Demais mudanças que não tocam código nem teste.
<code>style</code>	Formatação, sem efeito sobre a lógica.



Issues

Sempre por formulário. São seis tipos em [.github/ISSUE_TEMPLATE/](#), cada um com campos obrigatórios e etiqueta aplicada automaticamente. A caixa de texto em branco está desligada de propósito.

Segredos

Nenhum valor real de credencial entra em arquivo versionado, em nenhuma hipótese: nem em exemplo, nem em comentário, nem em teste. Credencial vazada em commit permanece no histórico mesmo depois de removida da árvore de trabalho.

Onde uma skill nova deve morar

O critério está registrado: se o conhecimento da skill sobrevive a uma mudança de lugar dentro deste repositório, ele pertence ao repositório compartilhado de skills, e não a este. Uma skill que cita [dbt/minc](#) ou [plugins/cliente_*.py](#) mora aqui, porque quebra junto com uma refatoração daqui e o mesmo PR conserta as duas coisas.

O que é gerado e não se edita à mão

Caminho	Origem
dbt/minc/target/	Saída de <code>dbt run</code> e <code>dbt docs generate</code> .
docs-pages/src/_data/	Acervo produzido por <code>make docs-collect</code> .
data/	Extrações brutas geradas por quem executa as DAGs.
requirements.generated.txt	Exportação do Poetry feita por <code>make setup</code> .

No site de documentação, número errado se corrige no coletor; explicação errada se corrige em [docs-pages/src/dominios.yml](#).





Cinco decisões de arquitetura estão registradas em `docs/adr/`, no formato de registro de decisão. Elas existem para que uma discussão já encerrada não volte à pauta sem informação nova.

ADR	Decisão	Data
0001	Formulários de issue em YAML, com campos obrigatórios e etiqueta preenchida, no lugar dos modelos Markdown herdados.	13/08/2026
0002	Skills versionadas no próprio repositório, e não instaladas por marketplace. Quem clona recebe todas, sem passo de instalação; em troca, correção feita na origem não chega sozinha.	13/08/2026
0003	Etiquetas organizadas em quatro eixos independentes, e não numa lista plana.	13/08/2026
0004	A revisão semanal apura antes de narrar: a skill é dividida em duas metades com responsabilidades separadas.	13/08/2026
0005	A ingestão do SALIC passa a ser feita pelo Trino, em fatias de chave: o dado não passa por Python — o motor lê do SQL Server e grava no Postgres, e o Airflow apenas emite SQL e registra o resultado.	24/08/2026

QUANDO ESCREVER UM ADR NOVO

A regra adotada é registrar uma decisão por discussão que tenha custado debate. O registro descreve o contexto, a decisão e as alternativas descartadas, e é isso que permite reabrir a questão no futuro sabendo o que já foi considerado.





Quem for estender a plataforma vai esbarrar nestas situações. Elas estão registradas aqui para que sejam encontradas antes, e não durante.

1. A automação não reprova o que deveria

A etapa de verificação de estilo termina com `|| true`, por decisão registrada, e cobre apenas SQL. Formatação, estilo e tipagem de Python são verificados só localmente. A suíte de testes mede cobertura sem limite mínimo.

2. Há código sem uso em plugins

Parte dos módulos de `plugins/` foi herdada de outro repositório da mesma família e não é importada por nenhuma DAG deste projeto. Antes de estender um cliente existente, confirme que ele é de fato consumido, para não investir em código que não executa.